Polymorphisme

De l'extérieur, on ne manipule que des références ou des pointeurs vers DinoEntity

Mais en réalité, cela peut rediriger vers des logiques différentes DinoAnimal & DinoPlayer

```
class Base {
    virtual void methode() = 0;
};
```

Cette méthode peut être surchargée

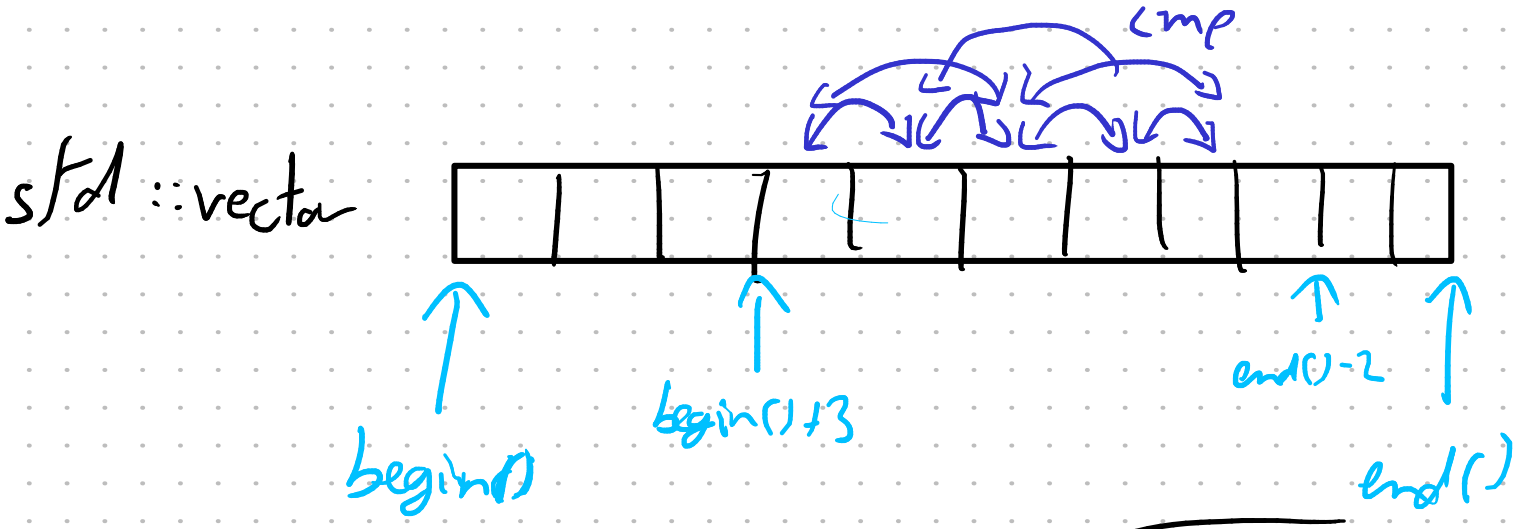
Cette méthode est purement virtuel = Pas d'implémentation par défaut

("Base est une classe abstraite" si et seulement si elle possède au moins une méthode purement virtuelle)

```
class Derivee : public Base {
    void methode() override;
};
```

On surcharge une méthode de la classe dont on hérite

(mot-clé pas obligatoire, mais bonne pratique de le mettre pour la lisibilité du code)

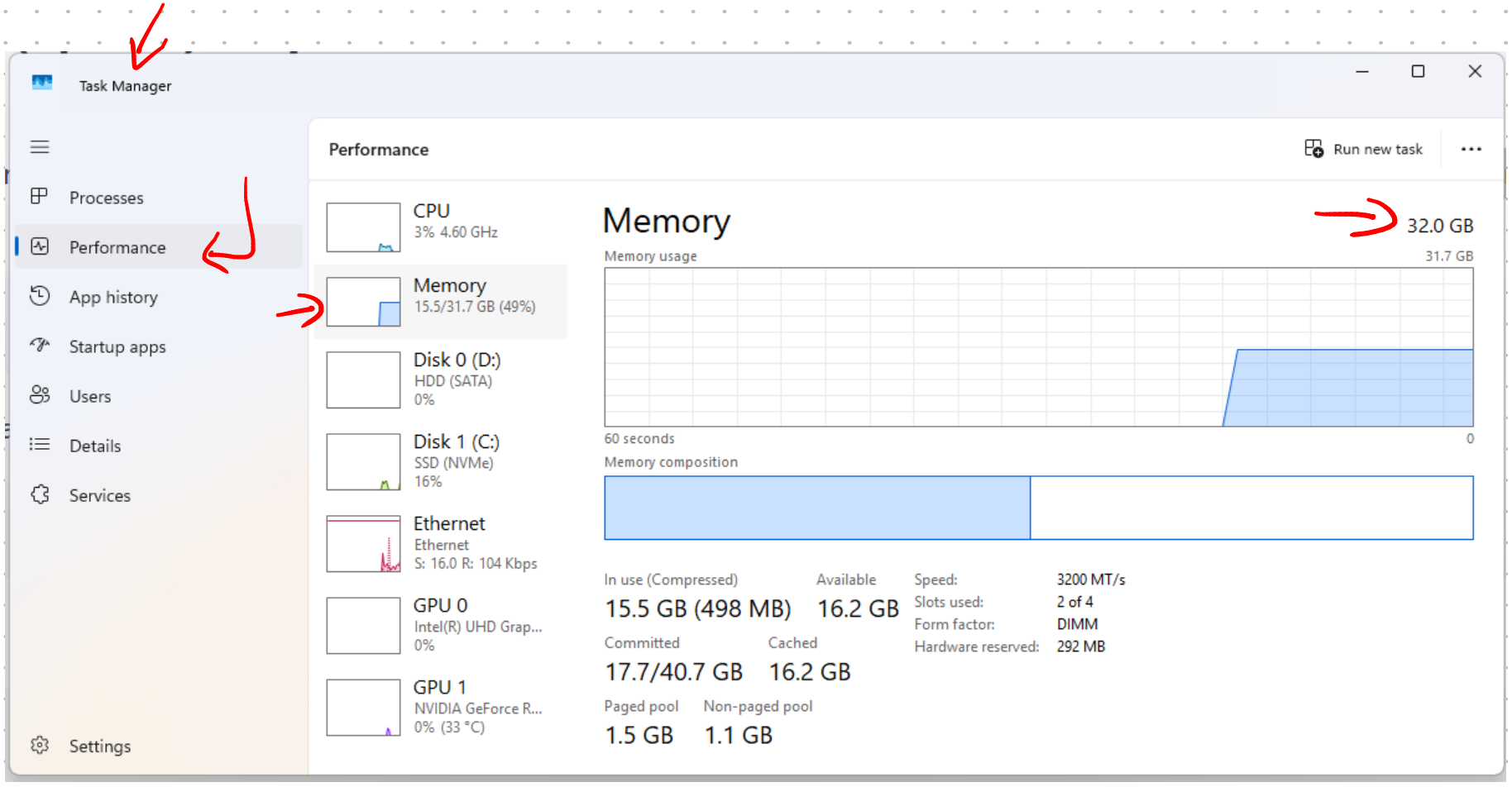


Itérateur
= Généralisation
d'un pointeur

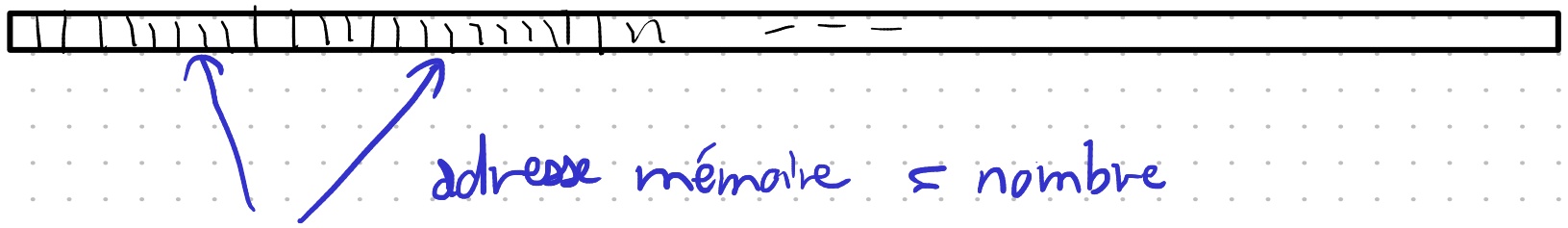
```
std::sort(      ,      , cmp    )

std::sort( begin()+3, end()-2, cmp )
```

8a)


Task Manager
Processes
Performance
App history
Startup apps
Users
Details
Services
Settings
Performance
Run new task
CPU
3% 4.60 GHz
Memory
15.5/31.7 GB (49%)
Disk 0 (D:)
HDD (SATA)
0%
Disk 1 (C:)
SSD (NVMe)
16%
Ethernet
Ethernet
S: 16.0 R: 104 Kbps
GPU 0
Intel(R) UHD Grap...
0%
GPU 1
NVIDIA GeForce R...
0% (33 °C)
Memory
32.0 GB
Memory usage
31.7 GB
60 seconds
0
Memory composition
In use (Compressed)
15.5 GB (498 MB)
Available
16.2 GB
Committed
17.7/40.7 GB
Cached
16.2 GB
Paged pool
1.5 GB
Non-paged pool
1.1 GB
Speed:
3200 MT/s
Slots used:
2 of 4
Form factor:
DIMM
Hardware reserved:
292 MB

Mémoire = Séquence d'octets, où chaque octet est identifié par un nombre



adresse mémoire = nombre

CPU

CPU 32-bits / 64-bits ⟹ adresses mémoire sont stockées sur 32 bits / 64 bits

"Combien d'octets adressables ?" → "Combien d'adresse mémoire différentes ?"

+1 bit → +1 bit → +1 bit

1 bit — 2 bits — 3 bits — 4 bits

0 = 0

00 = 0, 01 = 1

000 = 0, 001 = 1, 010 = 2, 011 = 3, 100 = 4, 101 = 5, 110 = 6, 111 = 7

0000 = 0, 0001 = 1, 0010 = 2, 0011 = 3, 0100 = 4, 0101 = 5, 0110 = 6, 0111 = 7

1 = 1

10 = 2, 11 = 3

100 = 4, 101 = 5, 110 = 6, 111 = 7

1000 = 8, 1001 = 9, 1010 = 10 = A, 1011 = 11 = B, 1100 = 12 = C, 1101 = 13 = D, 1110 = 14 = E, 1111 = 15 = F

Hexadécimal

×2 valeurs

×2 valeurs

×2 valeurs

1 bit → 2 = 2^1 valeurs

2 bits → 4 = 2^2

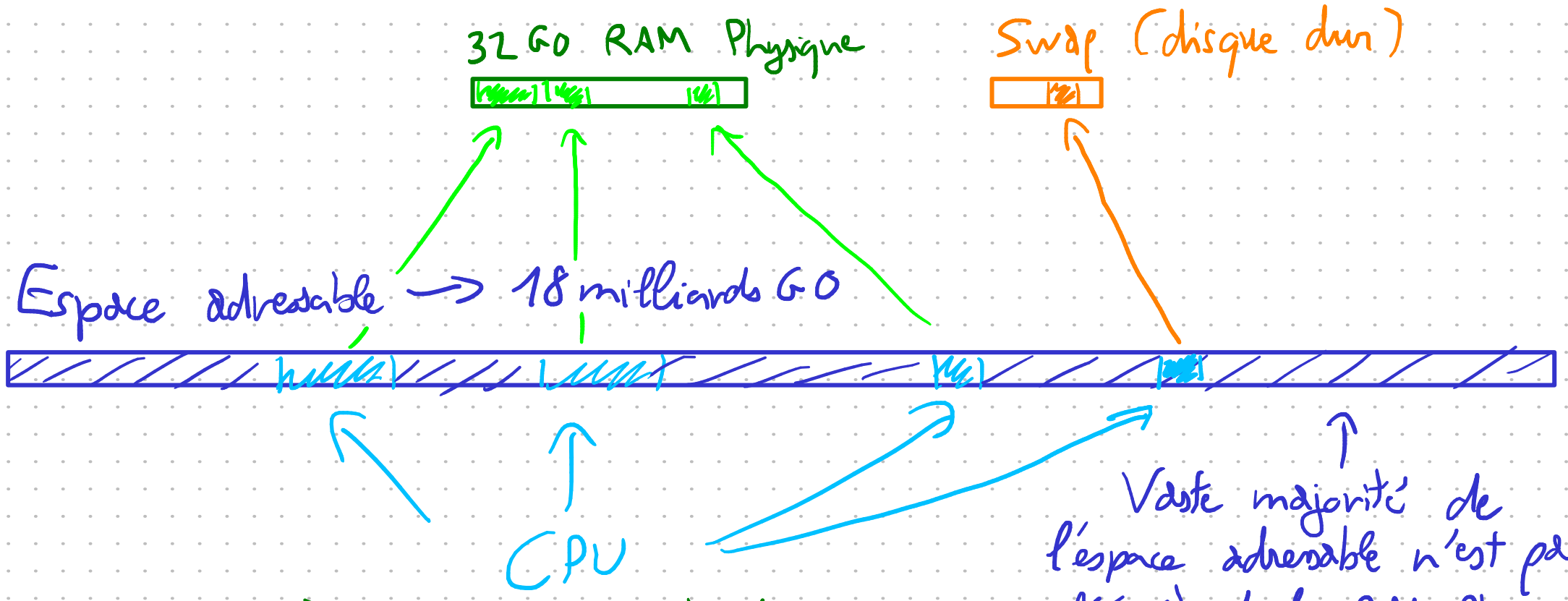
3 bits → 8 = 2^3

4 bits → 16 = 2^4

n bits → 2^n valeurs

→ CPU 32 bits → 2^{32} adresses possibles → 4 Go

→ CPU 64 bits → 2^{64} adresses possibles → 18 milliards Go



Vaste majorité de l'espace adressable n'est pas relié à de la RAM Physique

→ Si le CPU y accède Alors le programme crashe

En C++, on peut facilement accéder à des adresses invalides

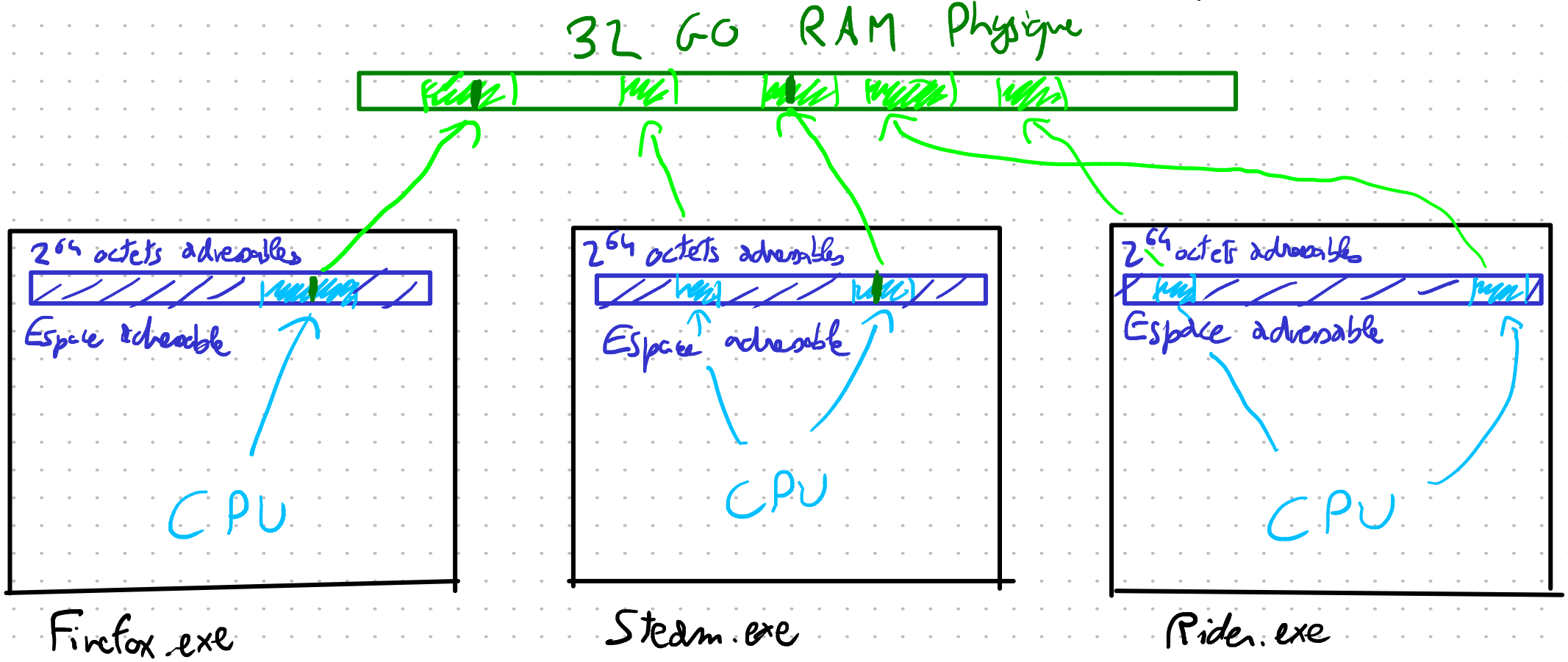
```
int v = *(int*)(12345678)
```

Lecture du contenu

Conversion en pointeur

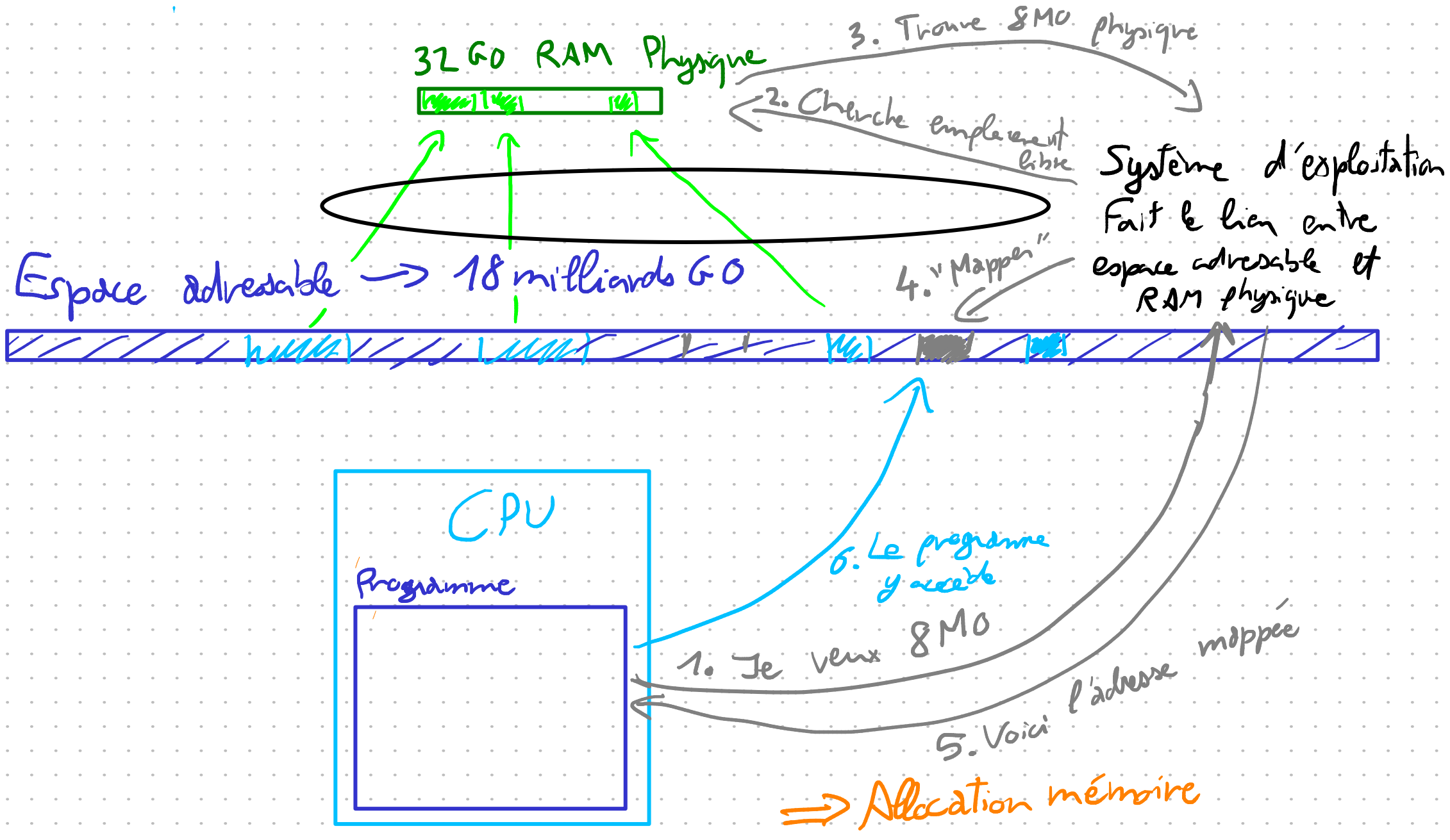
Nombre

Plusieurs programmes en même temps



Même si des programmes différents accèdent à la <u>même</u> adresse,

→ ce sera des emplacements <u>différents</u> en RAM physique.


32 GO RAM Physique
3. Trouve 8MO physique
2. Cherche emplacement libre
Système d'exploitation
Fait le lien entre espace adressable et RAM physique
Espace adressable → 18 milliards GO
4. "Mapper"
CPU
Programme
6. Le programme y accède
1. Je veux 8MO
5. Voici l'adresse mappée
⇒ Allocation mémoire

Mémoire = Séquence d'octets, où chaque octet est identifié par un nombre

En C++, on travaille avec des entiers,
des floats,
des structures,
des classes...

et on ne réfléchit pas aux octets

ABSTRACTION : SYSTÈME DE TYPES

En C++, il existe un système de type statique

→ Le compilateur sait toujours comment interpréter les octets dans la mémoire.

int v = 5; → Je trouve 4 octets pour stocker ma variable

int v2 = v + 10; → Je lis 4 octets en considérant que ça représente un entier.

TYPE = Comment interpréter une séquence d'octets

00111101110011001100110011001101

float

0.1f

uint32_t

1036831949

Color

R = 64
V = 204
B = 204
A = 205

Cyan translucide

POINTEUR = Adresse mémoire + Type

Adresse mémoire : dynamique
= nombre qui dépend de l'exécution du programme

Type : statique
= connu à la compilation

float* → Adresse vers 4 octets qui représentent un nombre réel

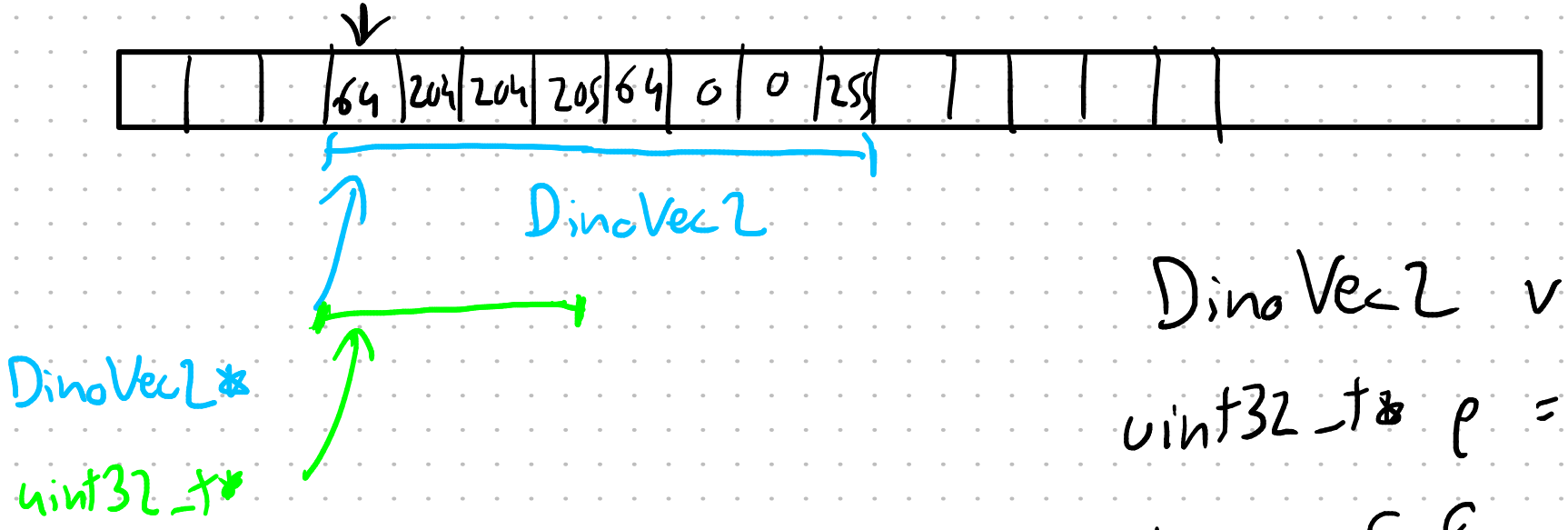
uint32_t* → Adresse vers 4 octets qui représentent un entier non-signé

DinoColor* → Adresse vers 4 octets qui représentent une couleur

```
float* pFloat = ___;
uint32_t* pUint = (uint32_t*) pFloat;
```

← Réinterprétation

Adresse mémoire = Numéro du premier octet du type



Obtient un pointeur

```
DinoVec2 v = {—};
uint32_t* p = (uint32_t*)&v;

v.x = 5.f;
v.y = 10.f;
*p = 30;
```

Déréférence

8d)

```
struct DinoVec2 {
    float x;
    float y;
};
```

Une structure contient toutes ses variables membres

Taille en octets de DinoVec2

= 2 × Taille en octets de Float

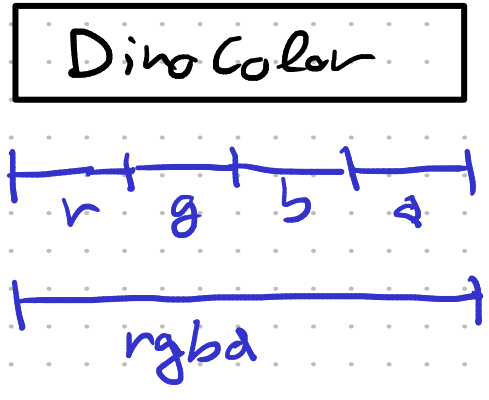
sizeof(DinoVec2) == sizeof(Float) x2

```
union DinoColor {
    struct {          4 octets
        uint8_t r;    1 octet
        uint8_t g;    1 octet
        uint8_t b;    1 octet
        uint8_t a;    1 octet
    };

    uint32_t rgba;    4 octets
};
```

Même emplacement mémoire

Une union contient l'une de ses variables membres.

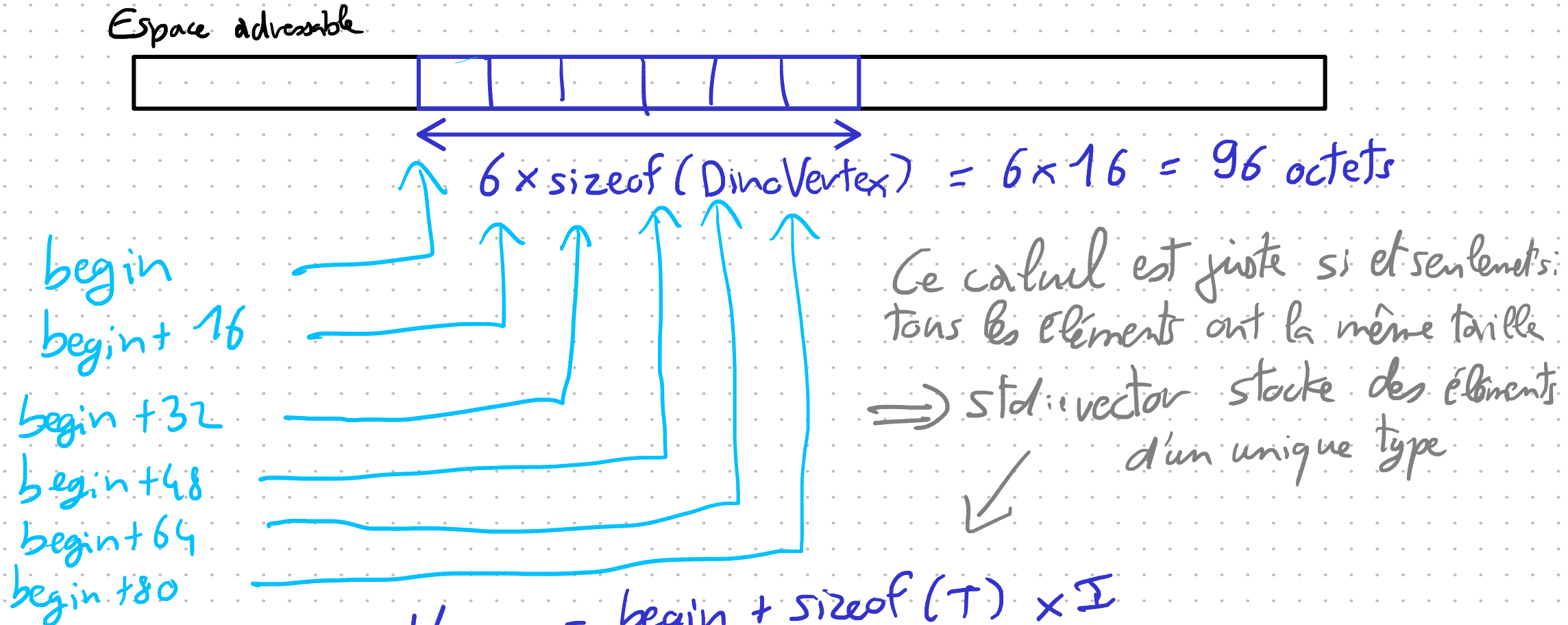


```
struct DinoVertex {
    DinoVec2 pos{};
    uint16_t u = 0;
    uint16_t v = 0;
    DinoColor color = DinoColor_WHITE;
};
```

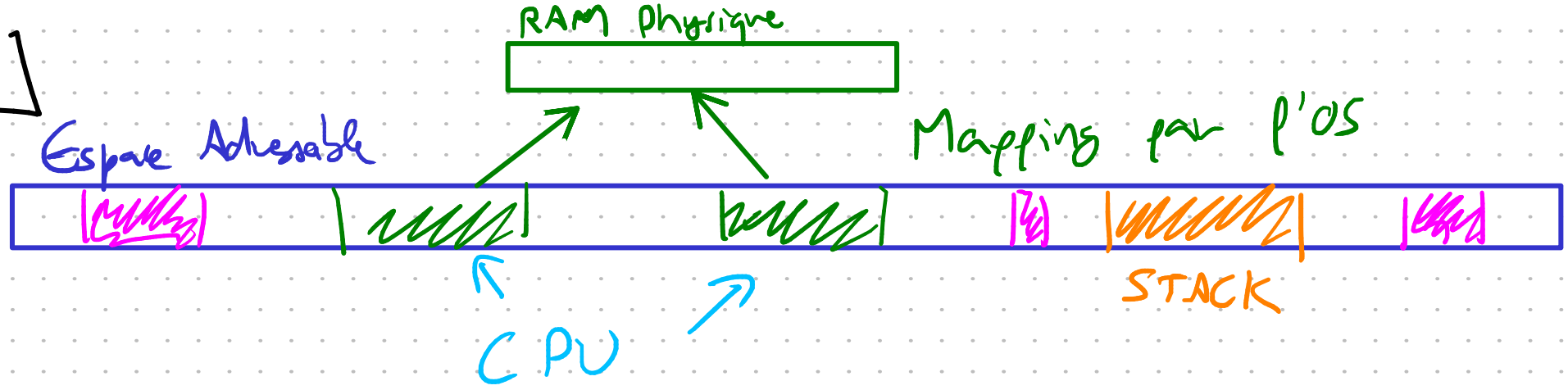
16 octets
8 octets
2 octets
2 octets
4 octets

8e)

```
std::vector<DinoVertex> vs;
vs.resize(6);
```



8h



Variables globales : elles sont connues à la compilation

→ Dans le fichier EXE, il est écrit

" le programme a besoin de N octets pour les var globales "

→ L'OS va allouer cette zone AVANT d'appeler la fonction main

Variables locales : (stack) allouée par l'OS avant la fonction main

→ Le compilateur génère du code pour sous-allouer les variables locales dans la stack

std::vector → Taille dynamique → Allocation mémoire via l'OS